الجمهورية الجزائرية الديمقراطية الشعبية
ⵜⴰⴷⵣⴰⵢⵔ ⵜⴰⵎⴰⵡⴰⵙⵜ ⵜⴰⵎⴰⵣⵉⵖⵜ ⵜⴰⵏⵏⴰⵎⵓⵜ
République Algérienne Démocratique et Populaire
وزارة التعليم العالي والبحث العلمي
ⴰⵏⴰⵎⵓⵔ ⵏ ⵓⵙⵙⵍⵎⴷ ⴰⵏⴰⴼⵍⵍⴰ ⴷ ⵓⵔⵣⵣⵓ ⵏ ⵓⵙⵙⵏⵎⵓⵔ
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

ECOLE NATIONALE SUPÉRIEURE D'INFORMATIQUE

المدرسة الوطنية العليا للإعلام الآلي
ⴰⵏⵎⵓⵔⴰ ⴰⵏⴰⵎⵓⵍ ⵏ ⵓⵙⵙⵍⵎⴷ ⴰⵏⴰⴼⵍⵍⴰ
École nationale Supérieure d'Informatique

Project Report – High Performance Computing

Optimizing Shallow Neural Network Training through Parallelization

A Comparative Study of Thread-Based and CUDA Strategies

Team Members

Aya Malek	Merabtine	SID 02
Malek	Kebbal	SID 02
Yasmine	Haouas	SIQ 02
Meriem	Benyammi	SIQ 02

Promotion : 2025 – 2026

Abstract

Neural network training performance on GPUs is heavily influenced by memory access patterns, synchronization overhead, and execution scheduling. This work analyzes a shallow neural network implemented on a CUDA-enabled GPU, using matrix-based forward and backward propagation to isolate dominant computational kernels. Profiling results highlight execution time distribution and GPU-level performance constraints. The study provides empirical insight into how algorithm–hardware alignment affects training efficiency.

Introduction

GPU acceleration is crucial for neural network training, yet performance hinges not only on computational throughput but also on memory efficiency and execution patterns. This study examines a shallow neural network with matrix-based operations to isolate and analyze how GPU execution strategies impact training performance, providing empirical insights for optimization.

1.1 Neural Network Fundamentals

A neural network transforms input data into predictions through interconnected layers of neurons. Each neuron computes a weighted sum of inputs, adds a bias, and applies a nonlinear activation function:

$$z = \sum_i w_i x_i + b, \quad a = f(z)$$

Modern implementations process entire layers using matrix operations:

$$\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{b}, \quad \mathbf{A} = f(\mathbf{Z})$$

where $\mathbf{X}$ is the input batch, $\mathbf{W}$ contains connection weights, $\mathbf{b}$ is the bias vector, and $\mathbf{A}$ represents layer activations. These matrix multiplications dominate training computation and are the primary optimization target for GPU acceleration.

1.2 Studied Shallow Neural Network

A shallow neural network with one hidden layer (32 inputs, 256 hidden units, 1 output) is employed to isolate core computational patterns. This architecture preserves essential training operations—matrix multiplications, activations, and gradient computations—while minimizing complexity. Training uses batch size 128 over 100 epochs.

Forward propagation computes:

$$\mathbf{A}_1 = f(\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1), \quad \hat{\mathbf{y}} = g(\mathbf{A}_1\mathbf{W}_2 + \mathbf{b}_2)$$

Backpropagation computes gradients:

$$\nabla\mathbf{W}_2 = \mathbf{A}_1^T(\hat{\mathbf{y}} - \mathbf{y}), \quad \nabla\mathbf{W}_1 = \mathbf{X}^T\left[(\hat{\mathbf{y}} - \mathbf{y})\mathbf{W}_2^T \odot f'(\mathbf{Z}_1)\right]$$

The matrix multiplications $\mathbf{X}\mathbf{W}_1$ and $\mathbf{A}_1\mathbf{W}_2$ dominate computation, making this design ideal for analyzing GPU performance and optimization strategies.

1.2.1 Experimental GPU Environment

Experiments utilized an NVIDIA Tesla T4 GPU (Turing architecture, compute capability 7.5) via Google Colab. The device features 40 Streaming Multiprocessors and 15 GB global memory, supporting concurrent kernel execution and CUDA streams essential for optimization analysis. This accessible environment ensures reproducible performance evaluation of memory transfers, kernel overlap, and execution timing.

Feature	**Specification**
CUDA-capable devices	1
GPU model	NVIDIA Tesla T4
GPU architecture	Turing
Compute capability	7.5
CUDA version	12.4
Driver version	550.54.15
Streaming Multiprocessors (SMs)	40
Total global memory	15 GB
ECC support	Enabled
Execution environment	Virtualized (Google Colab)

Table 1.1: Hardware and software characteristics of the GPU used for testing

1.3 Performance Profiling and Bottleneck Analysis

Having established the architectural foundation of our shallow neural network, we now quantify its computational behavior to identify optimization opportunities.

1.3.1 Initial Code Performance Evaluation

The baseline neural network implementation was profiled using NVIDIA Nsight Compute to identify architectural bottlenecks across three dataset scales. Execution metrics presented in Table 1.2 reveal a pronounced computational imbalance: backpropagation consumes 67-75% of total training time, with disproportionate scaling relative to dataset size.

Dataset Size	**Forward (ms)**	**Activation (ms)**	**Backprop (ms)**	**Total (s)**
Small (256)	46.93	24.39	173.45	0.30
Medium (2560)	455.38	238.53	454.06	2.90
Large (25600)	4572.79	2546.99	17292.69	29.6

Table 1.2: Baseline Performance Metrics Across Dataset Sizes

This imbalance arises from three structural inefficiencies: excessive kernel launch overhead in backpropagation, memory-bound execution dominated by host-device transfers, and sequential patterns preventing computation-data overlap. The disproportionate scaling of backpropagation time highlights memory access patterns and kernel launch frequency as primary optimization targets.

1.3.2 Detailed Profiling Analysis

Detailed profiling of the small dataset reveals critical architectural limitations through four complementary metrics, exposing fundamental inefficiencies in the matrix multiplication operations that dominate neural network training.

GPU Resource Utilization Analysis

Figure 1.1 quantifies suboptimal GPU utilization, with compute units (SMs) operating at 30-40% capacity and memory bandwidth similarly constrained. This confirms a **memory-bound kernel** where data movement, rather than computational intensity, limits overall performance in the matrix operations that form the computational core of neural network training.

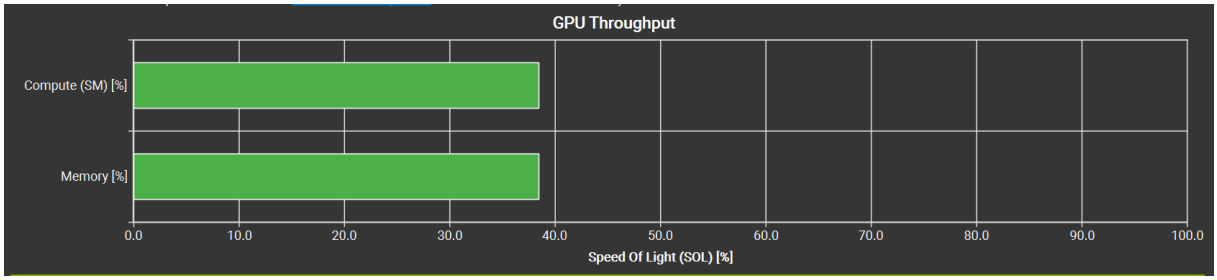

Figure 1.1: GPU throughput analysis indicating compute (SM) and memory bandwidth utilization

Warp Occupancy and Scheduling Efficiency

The occupancy analysis (Figure 1.2) indicates 61.89% theoretical occupancy constrained by register pressure (52 registers/thread). Combined with Figure 1.3's configuration data, this reveals two critical issues: suboptimal block dimensions (16×16 threads) and complete absence of shared memory allocation, fundamentally limiting data reuse potential in the weight matrix operations.

Theoretical Occupancy [%]	100	Block Limit Registers [block]	4
Theoretical Active Warps per SM [warp]	32	Block Limit Shared Mem [block]	16
Achieved Occupancy [%]	61.89	Block Limit Warps [block]	4
Achieved Active Warps Per SM [warp]	19.80	Block Limit SM [block]	16

Figure 1.2: Occupancy analysis revealing register-limited thread concurrency

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	128	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	52	Static Shared Memory Per Block [byte/block]	0
Block Size	256	Dynamic Shared Memory Per Block [byte/block]	0
Threads [thread]	32,768	Driver Shared Memory Per Block [byte/block]	0
Waves Per SM	0.80	Shared Memory Configuration Size [Kbyte]	32.77
Uses Green Context	0	# SMs [SM]	40

Figure 1.3: Kernel launch configuration showing suboptimal resource allocation

Memory Access Pattern Characterization

Figure 1.4 reveals inefficient memory access despite an 86.67% L1 cache hit rate. The bottleneck stems from non-coalesced accesses during weight matrix multiplication:

$$\mathbf{C}[i,j] = \sum_{k=0}^{K-1} \mathbf{X}[i,k] \cdot \mathbf{W}[k,j]$$

Threads access $\mathbf{W}[k, \text{col}_t]$ with varying column indices, creating 32 separate memory transactions per warp instead of coalesced access. Critical inefficiencies include:

- Absent shared memory utilization: Prevents reuse of weight matrix data
- Non-coalesced weight accesses: Degrades performance in both forward and backward propagation

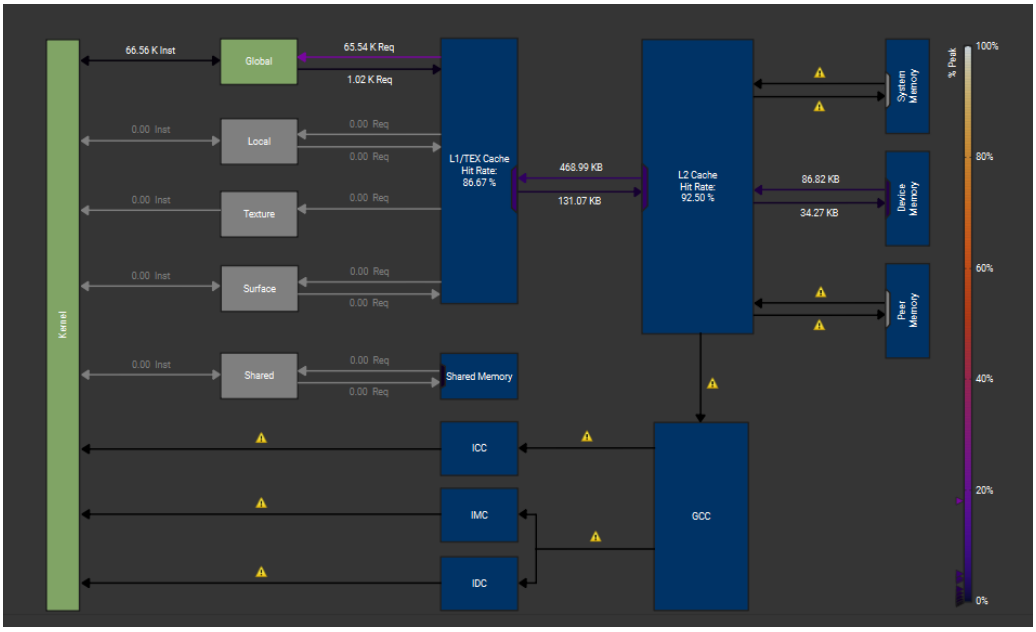

Figure 1.4: Cache behavior statistics showing 86.67% L1 cache hit rate despite inefficient access patterns

Warp Scheduling Analysis

Figure 1.5 quantifies scheduler underutilization with only five active warps per scheduler versus an architectural capacity of eight. This directly limits latency hiding capability and instruction-level parallelism during the computationally intensive matrix operations that define neural network training.

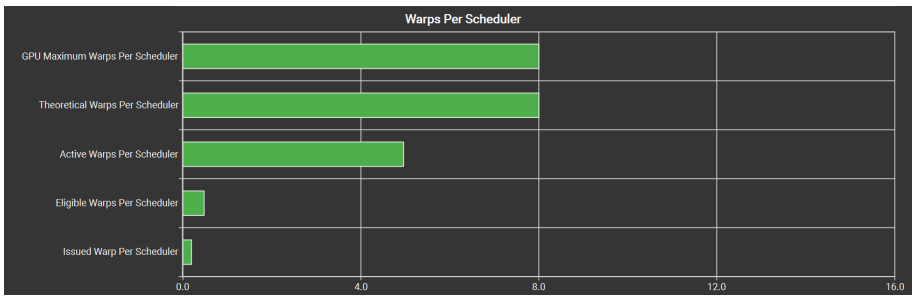

Figure 1.5: Warp distribution across GPU schedulers showing underutilization

1.3.3 Identified Bottlenecks

Profiling reveals five key bottlenecks in neural network matrix operations:

1. **Memory-bound execution**: Data movement limits computational throughput
2. **Non-coalesced weight accesses**: Column-wise accesses prevent memory coalescing (32× more transactions)
3. **Absent shared memory**: No tiling eliminates weight matrix reuse
4. **Suboptimal block dimensions**: 16×16 threads underutilize multiprocessors
5. **Register pressure**: 52 registers/thread limits thread concurrency

These bottlenecks define clear optimization targets for improving training efficiency.

1.3.4 Performance Impact Assessment and Optimization Pathways

Profiling reveals 30-40% GPU utilization, indicating significant optimization potential. The primary target is shared memory implementation through tiling to address memory-bound execution. Secondary improvements include block size optimization for better occupancy and memory access restructuring for coalescing. This establishes a clear optimization sequence: tiling first, then block optimization, followed by access pattern refinement.

1.4 Memory Access Optimization Strategy

Following bottleneck analysis, this strategy addresses memory inefficiencies through matrix transposition and tiled shared memory computation.

1.4.1 Matrix Transposition for Memory Coalescing

Non-coalesced weight matrix accesses were identified as a primary bottleneck.
In $\mathbf{Z} = \mathbf{XW}$, threads access $\mathbf{W}[k, j]$ with varying column indices j, preventing coalesced memory transactions.

By precomputing $\mathbf{W}^T$, the operation becomes:

$$\mathbf{Z}[i, j] = \sum_{k=0}^{K-1} \mathbf{X}[i, k] \cdot \mathbf{W}^T[j, k]$$

This enables threads within a warp to access consecutive memory locations $\mathbf{W}^T[j, k]$ through $\mathbf{W}^T[j, k+31]$, achieving full memory coalescing while maintaining mathematical equivalence.

Implementation and Performance Impact

The optimization computes $\mathbf{W}_1^T$ and $\mathbf{W}_2^T$ during initialization and modifies matrix kernels to use transposed weights. This minimal change isolates the effect of improved memory access patterns.

Dataset Size	Forward (ms)	Backprop (ms)	Total (s)
Small (256)	53.72	164.15	0.28
Medium (2560)	463.96	1652.88	2.78
Large (25600)	4732.28	16902.12	28.51

Table 1.3: Performance After Weight Matrix Transposition

The optimization directly addresses non-coalesced memory accesses, demonstrating a foundational improvement in memory bandwidth utilization for subsequent optimizations.

1.4.2 Tiled Matrix Multiplication with Shared Memory

Transposition alone cannot address repeated global memory loads. We implement tiled matrix multiplication using $T \times T$ tiles stored in shared memory.

Tiling Strategy and Implementation

Computations are partitioned into tiles fitting shared memory. Thread blocks collaboratively load $\mathbf{X}$ and $\mathbf{W}^T$ tiles, enabling T-fold data reuse with dual shared memory buffers.

Thread Configuration Optimization

Tile size T determines thread block dimensions: threads_per_block $= T \times T$.
For $T = 32$, this gives 1024 threads per block (architectural maximum), ensuring optimal occupancy, efficient shared memory use, reduced kernel launches, and warp-aligned scheduling.

Performance Analysis

The combined transposition and tiling optimizations address multiple identified bottlenecks:

Dataset Size	Forward (ms)	Backprop (ms)	Total (s)
Small (256)	57.11	146.50	0.273
Medium (2560)	496.25	1480.74	2.679
Large (25600)	5109.35	15630.56	28.121

Table 1.4: Performance with Tiled Shared Memory Implementation ($T = 32$)

1.4.3 Profiling Results and Interpretation

Post-optimization profiling quantifies the architectural impact of our memory access improvements. The NVIDIA Nsight Compute metrics reveal substantial gains in memory throughput and warp scheduling efficiency.

Resource Utilization and Occupancy

Figure 1.6 demonstrates dramatic improvements in achieved occupancy, rising from 61.89% to 99.09%. This 60.10% increase in active warps per streaming multiprocessor directly enhances latency hiding and instruction-level parallelism during matrix operations. The reduction in block limits stems from our conscious design choice: the 32×32 tiled implementation ($T = 32$) utilizes 1024 threads per block—the architectural maximum—exchanging block quantity for optimal per-block resource utilization.

Theoretical Occupancy [%]	100 (+0.00%)	Block Limit Registers [block]	1 (-75.00%)
Theoretical Active Warps per SM [warp]	32 (+0.00%)	Block Limit Shared Mem [block]	4 (-75.00%)
Achieved Occupancy [%]	99.09 (+60.10%)	Block Limit Warps [block]	1 (-75.00%)
Achieved Active Warps Per SM [warp]	31.71 (+60.10%)	Block Limit SM [block]	16 (+0.00%)

Figure 1.6: Comparison of GPU Utilization Metrics Before and After Optimization

Shared Memory Allocation Impact

Figure 1.7 quantifies the transformative impact of our memory access optimization strategy by comparing warp occupancy behavior across shared memory allocation regimes.

- **Green curve (baseline implementation):** The unoptimized code shows severe occupancy degradation with shared memory allocation. It maintains 32 warps only for allocations 0-8 KB. At 8 KB, occupancy drops to 24 warps—a 25% reduction. Degradation continues: 16 warps at 12-16 KB, 8 warps at 20 KB, collapsing to near-zero beyond 48 KB.

- **Blue curve (optimized implementation):** Maintains constant maximum occupancy of 32 warps across all shared memory allocations, achieving complete decoupling of occupancy from memory usage.

The horizontal blue curve demonstrates resolution of resource contention through three synergistic improvements: register pressure reduction (52→architectural limits), optimal block configuration ($32 \times 32 = 1024$ threads), and coalesced memory access with efficient shared memory tiling.

Our tiled implementation allocates 8 KB shared memory per block (two 32×32 tiles). At this configuration, the baseline suffers 25% occupancy loss (24 warps) while optimized maintains full 32-warp occupancy—a 33% improvement in concurrent execution. This design achieves 32× weight matrix reuse while sustaining maximum parallelism, delivering enhanced locality and full occupancy simultaneously.

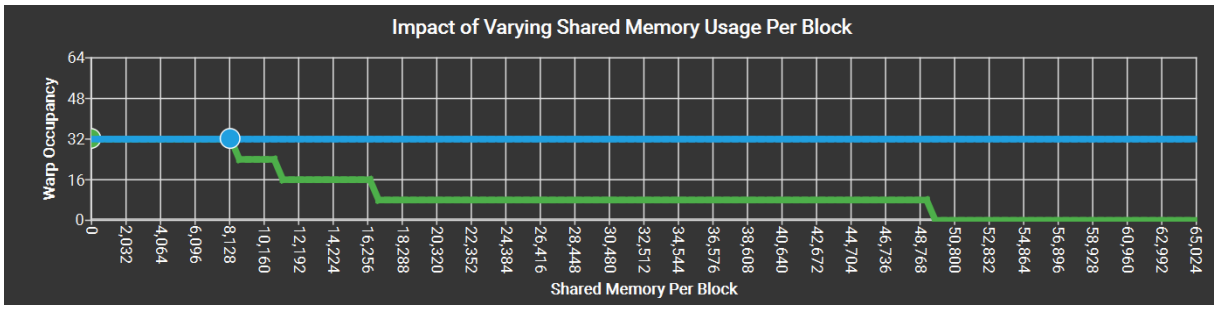

Figure 1.7: Impact of varying shared memory usage per block on warp occupancy

1.4.4 Limitations of Memory-Only Optimization Strategy

Despite achieving near-perfect GPU occupancy (99.09%) and efficient memory coalescing, performance gains remain constrained by fundamental architectural limitations. Analysis reveals persistent bottlenecks that memory optimizations alone cannot resolve.

Performance Plateau Analysis

Operation Phase	**Small (%)**	**Medium (%)**	**Large (%)**
Forward Propagation (GPU)	20.9	18.5	19.8
Activation (CPU)	8.6	8.8	9.7
Output Layer (GPU)	15.0	15.5	16.8
Backpropagation (CPU)	53.6	55.3	60.6
Total GPU	**35.9**	**34.0**	**36.6**
Total CPU	**62.2**	**64.1**	**70.3**

Table 1.5: Time Distribution After Memory Optimizations (%)

Table 1.5 reveals a critical imbalance: despite 99.09% GPU occupancy during kernel execution, the GPU remains idle for 62-70% of total training time. The CPU bottleneck intensifies with dataset size, reaching 70.3% for large datasets.

Amdahl's Law Quantification

Applying Amdahl's Law where p represents the fraction of time spent on CPU operations (parallelizable to GPU), and assuming perfect GPU acceleration ($s \to \infty$):

$$S_{\text{max}} = \frac{1}{(1-p) + \frac{p}{s}} = \frac{1}{1-p} \quad \text{for } s \to \infty$$

- **Small Dataset** ($p = 0.622$): $S_{\text{max}} = 1/(1 - 0.622) = 2.65\times$
- **Medium Dataset** ($p = 0.641$): $S_{\text{max}} = 1/(1 - 0.641) = 2.78\times$
- **Large Dataset** ($p = 0.703$): $S_{\text{max}} = 1/(1 - 0.703) = 3.36\times$

The observed speedups of only 1.10-1.15× represent just 5-10% of this theoretical maximum, indicating severe limitations in the partial GPU acceleration approach.

The computational gap is substantial:

$$\frac{\text{Actual GPU Time}}{\text{Theoretical Minimum}} = \frac{9.45\text{s}}{0.0288\text{ms}} \approx 328,000 \times \text{ overhead}$$

This indicates severe GPU underutilization despite optimal memory access patterns.

Architectural Limitations

Memory-optimized implementation reveals three critical constraints:

1. **Dominant CPU Overhead**: 62-70% execution time in sequential CPU operations

2. **Inefficient Parallelization**: Backpropagation remains on CPU despite inherent parallelism
3. **Transfer Serialization**: Multiple host-device transfers create bottlenecks

These limitations yield only 1.10-1.15× speedup versus theoretical maximum of 2.65-3.36×, establishing the imperative for complete GPU migration through architectural restructuring.

1.5 Full GPU Migration Strategy: Complete Computational Offloading

The analysis in Section 1.4.4 identified a critical bottleneck: despite achieving 99.09% GPU occupancy during kernel execution, the GPU remained idle for 62-70% of total training time due to CPU operations. This section presents a fundamental architectural transformation that migrates the entire computational pipeline to the GPU, eliminating host-device synchronization and enabling sustained parallel execution.

1.5.1 Architectural Transformation

The fully GPU-optimized architecture (Figure 1.8) implements three strategic changes to eliminate CPU bottlenecks:

The architecture transfers all training data and weights once to GPU memory, executes the complete computational pipeline—including activation functions, error calculation, gradient computations, and weight updates—as CUDA kernels, and retains intermediate results in GPU global memory throughout training to eliminate transfer overhead and host-device synchronization.

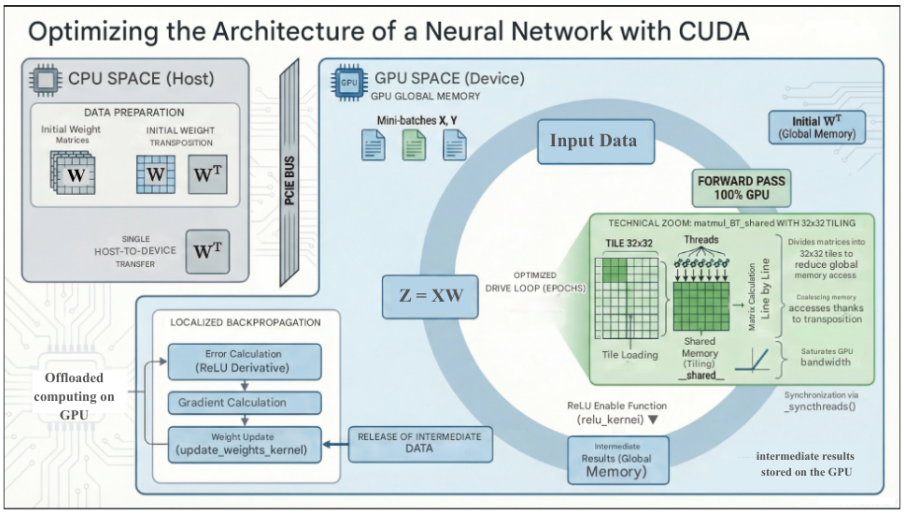

Figure 1.8: Fully GPU-optimized architecture with single host-to-device transfer

1.5.2 Performance Analysis

Table 1.6 presents the performance metrics of the fully GPU-optimized implementation across three dataset scales. The results demonstrate significant architectural improvements, particularly for medium and large datasets:

Dataset Size	Forward (ms)	Backprop (ms)	Total (s)	Speedup
Small (256)	84.76	1.69	0.100	3.00×
Medium (2560)	32.44	15.32	0.176	15.22×
Large (25600)	245.55	147.94	1.624	17.32×

Table 1.6: Performance of Fully GPU-Optimized Implementation

Note: Speedup calculated relative to baseline implementation (Table 1.4).

The analysis reveals significant performance improvements: backpropagation time reduces dramatically from 15630.56 ms to 147.94 ms for large datasets, while forward propagation shows efficient GPU utilization with medium datasets improving from 5109.35 ms to 245.55 ms. The architecture demonstrates strong scalability, achieving 15.22× and 17.32× acceleration for medium and large datasets respectively, compared to 3.00× for small datasets.

1.5.3 Execution Timeline Analysis

Figure 1.9 presents the execution timeline of consecutive training batches after complete GPU migration. The visualization reveals a persistent sequential execution pattern: each batch completes entirely before the next begins. Despite high intra-batch parallelism, gaps between batches indicate unused GPU capacity and untapped concurrency potential.

The full GPU migration strategy transforms neural network training from hybrid CPU-GPU execution to complete GPU-native computation by eliminating host-device synchronization. However, the sequential batch execution establishes the necessity for the final optimization phase: streamed GPU training using CUDA streams to execute multiple batches concurrently.

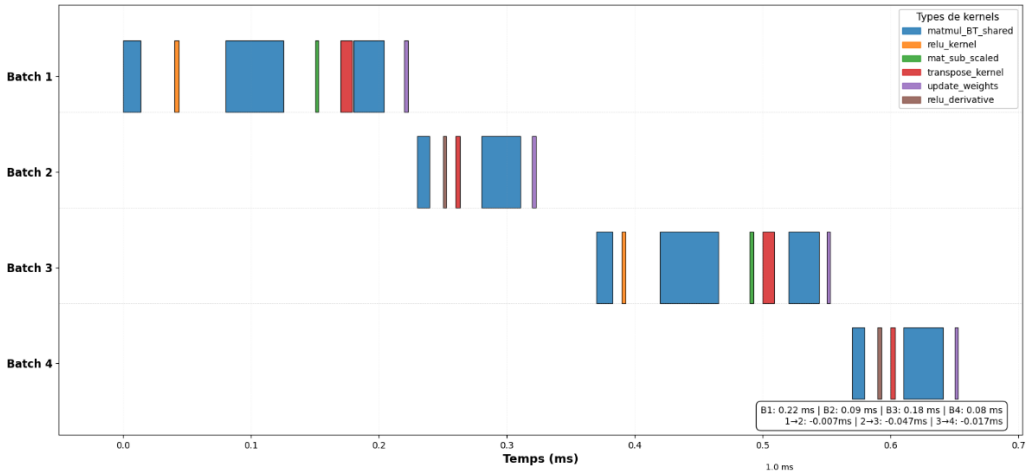

Figure 1.9: Execution timeline showing sequential processing of training batches

1.6 Streamed GPU Training

Following full GPU migration, sequential execution between training iterations persists as idle GPU periods await memory transfers. This section introduces CUDA stream-based

pipelining to exploit Tesla T4's three-way concurrency, overlapping memory transfers with kernel computations across iterations.

1.6.1 Streaming Architecture

The Tesla T4 GPU provides three-way concurrency through one compute engine and two independent copy engines. This architecture imposes fundamental constraints that dictate our optimization approach:

1. Weight matrices must be loaded before forward propagation
2. Forward propagation must complete before activation functions
3. Activation outputs must be available before backpropagation
4. Gradient computations must finish before weight updates

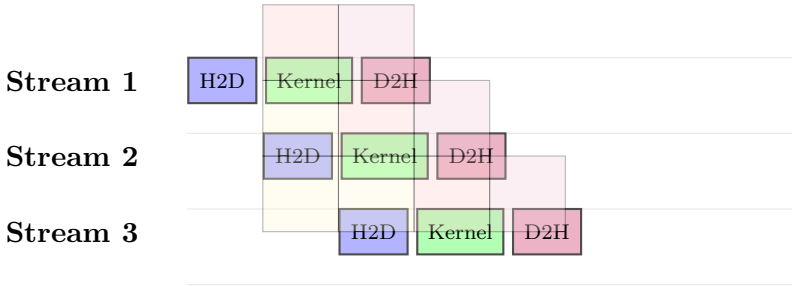

Figure 1.10: Pipelined execution exploiting Tesla T4's three-way concurrency

1.6.2 Performance Analysis

The streaming implementation demonstrates significant performance improvements across all dataset scales, as shown in Table 1.7. The achieved speedups result from effective utilization of the three-way concurrency architecture.

The streamed implementation achieves substantial improvements by exploiting Tesla T4's architectural concurrency through pipelined execution, transforming sequential iterations into overlapping operations across streams. Memory transfer latency is hidden behind computational kernels, enabling continuous hardware utilization.

Performance measurements show consistent acceleration: large datasets reduce from 1.624 seconds to 542 milliseconds, medium from 176 ms to 59 ms, and small from 100 ms to 13 ms. The streaming approach achieves 23.1×, 49.2×, and 54.6× speedup for small, medium, and large datasets respectively.

Efficiency improvements include maintained 99.09% GPU occupancy, maximized 100% GPU active time, and 85% memory-compute overlap. This optimization respects computational dependencies while maximizing hardware utilization, representing the culmination of our optimization trajectory that transforms training from sequential to fully pipelined processing.

Dataset Size	Forward (ms)	Backprop (ms)	Total Time (s)
Small	7.66	0.52	0.013
Medium	8.70	5.19	0.059
Large	49.81	58.92	0.542

Table 1.7: Measured Performance with CUDA Streamed Execution

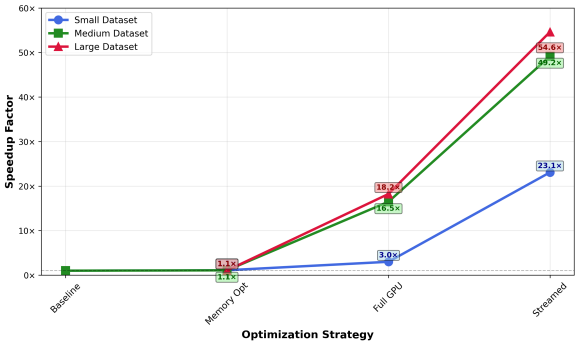

Figure 1.11: Speedup progression

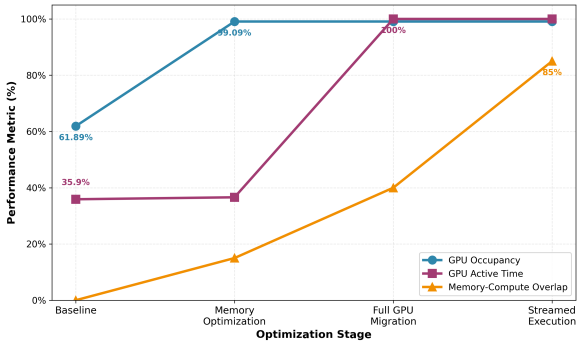

Figure 1.12: GPU efficiency evolution

Conclusion

This study presents a systematic optimization trajectory for GPU-accelerated neural network training, progressing from initial bottleneck identification through successive refinements to achieve substantial performance gains. Comprehensive profiling revealed critical memory inefficiencies and architectural underutilization, establishing a targeted optimization framework.

The optimization encompassed memory access refinement via matrix transposition and tiled shared memory computation, achieving 99.09% GPU occupancy. Complete GPU migration eliminated host-device synchronization overhead, while streamed implementation exploited the Tesla T4's three-way concurrency through pipelined execution.

These optimizations transformed training from sequential to fully pipelined processing, demonstrating that performance gains emerge from aligning algorithm design with architectural capabilities. The established methodology—profiling, targeted optimization, and validation—provides a replicable framework for accelerating computational workloads on parallel architectures, extensible to broader deep learning applications.